

# Pair A - Engine, Sensors, Observer

---

*Adapted from Yakov's Codebase Reading and Defence Guide (Sept 2026), split per pair by Henry. Full PDF: [docs/COMP2000\\_Traffic\\_Simulation\\_Codebase\\_Guide.pdf](#) .*

**You are:** Ali, Henry, Oscar. **You own:** the tick loop, safety readings, spawning logic, and the observer wiring that pushes updates to the UI. You are the coordinator of the whole simulation.

## Files you own

---

|                         |                                    |
|-------------------------|------------------------------------|
| src/trafficsim/         |                                    |
| Main.java               | <- entry point                     |
| SimConstants.java       | <- central tuning knobs            |
| engine/                 |                                    |
| SimulationEngine.java   | <- the tick loop                   |
| TrafficSensor.java      | <- collision-avoidance sensor      |
| SensorReading.java      | <- immutable per-vehicle reading   |
| SimulationObserver.java | <- observer interface              |
| Statistics.java         | <- streams-based metrics           |
| VehicleSpawner.java     | <- adds vehicles to boundary lanes |
| view/                   |                                    |
| SimulationDisplay.java  | <- observer target, drawing only   |
| MainFrame.java          | <- Swing window scaffolding        |

## What happens in one tick

---

`SimulationEngine.doStep` runs the following phases in order. Every driver decides from the same initial snapshot.

| Phase   | What happens                                                                 | Why this order                                 |
|---------|------------------------------------------------------------------------------|------------------------------------------------|
| Lights  | Every intersection advances one state timer.                                 | Vehicles see the current signal state.         |
| Collect | Engine gathers vehicles from every lane.                                     | One set of active objects for this tick.       |
| Sense   | <code>TrafficSensor</code> builds a <code>SensorReading</code> per vehicle.  | All drivers decide from the same snapshot.     |
| Move    | Each vehicle updates speed and position.                                     | Movement uses the precomputed safe distance.   |
| Despawn | Vehicles outside the enlarged map are removed.                               | Map does not fill with invisible old vehicles. |
| Spawn   | A new vehicle may appear if below <code>SPAWN_CAP</code> .                   | Traffic stays populated but bounded.           |
| Notify  | Display receives <code>onSimulationStep()</code> -> <code>repaint()</code> . | UI redraws only after a complete tick.         |

## Timing and thread safety

`run()` starts a daemon thread named `sim-loop`. At 30 Hz one tick has ~33,333,333 ns.

```
long tickNanos = 1_000_000_000L / Math.max(1, tickRate);
nextTick += tickNanos;
long waitNanos = nextTick - System.nanoTime();
if (waitNanos > 0) LockSupport.parkNanos(waitNanos);
else nextTick = System.nanoTime();
```

Both the engine and `SimulationDisplay` synchronise on the network. That prevents the UI from painting one vehicle before a move and another after a move in the same frame.

## Sensors and collision avoidance

`SensorReading` is an immutable Java **record** with two distances: `distToVehicleAhead` and `distToRedStop`. Positive infinity means no relevant obstacle.

```
public double effectiveStopDistance() {
    return Math.min(distToVehicleAhead, distToRedStop);
}
```

## Same-lane spacing

`TrafficSensor.nearestVehicleAhead` converts centre-to-centre distance into safe bumper-to-bumper free space:

```

centreToCentre = signedDistanceAhead(self, other.x, other.y);
noseToTail = centreToCentre
    - self.length / 2
    - other.length / 2
    - safetyGap;

```

Example: centres 30 units apart, both vehicles 18 units long, safety gap 4 -> free space is `30 - 9 - 9 - 4` = 8 units.

## Signed distance via dot product

```

double dx = pointX - vehicleX;
double dy = pointY - vehicleY;
return dx * direction.dx() + dy * direction.dy();

```

For an eastbound vehicle, direction is `(1,0)` so the result is simply `dx` : positive is ahead, negative is behind. Same trick works for all four directions without separate logic.

## Intersection checks

| Sensor check                                 | What it prevents                                                             |
|----------------------------------------------|------------------------------------------------------------------------------|
| <code>nearestSignalStopAhead</code>          | Normal cars stop before red / yellow.                                        |
| <code>nearestBlockedIntersection</code>      | Car with green does not enter if stopped traffic beyond would block the box. |
| <code>nearestOccupiedIntersectionStop</code> | Vehicle waits if perpendicular traffic is inside the junction.               |
| <code>nearestEmergencyPriorityStop</code>    | Normal traffic yields to a clear-path, siren-on emergency vehicle.           |

## Spawning (VehicleSpawner)

Weighted mixes over many spawns:

| Vehicle   | Weight | Long-run share |
|-----------|--------|----------------|
| Car       | 75     | 75%            |
| Truck     | 12     | 12%            |
| Bus       | 6      | 6%             |
| Emergency | 7      | 7%             |

Boundary-lane selection: for each road lane, `laneEntryPoint` finds the map edge. Spawner keeps only points on the bounding box, shuffles, and picks the first non-crowded one.

## Observer wiring (yours + view/)

---

`SimulationDisplay` implements `SimulationObserver`. Engine adds it as an observer at construction. After a complete tick, engine calls `notifyObservers()` -> `display.onSimulationStep()` -> `repaint()`. Swing then invokes `paintComponent`.

**The renderer never changes vehicle positions.** It only reads state.

## Where you touch other pairs

---

- **Pair B (Network):** You call `network.getRoads()`, `road.getLanes()`, `lane.getVehicles()`. Recent change: `getVehicles()` now returns an unmodifiable view - use `lane.removeVehicle(v)` in a loop rather than `removeIf` on the returned list.
- **Pair C (Vehicles):** You call `vehicle.move(reading)`. Vehicles hold their own state; you don't set position directly.
- **Pair D (Lights):** You call `intersection.updateSignals()` in the Lights phase. Read state via `light.phaseFor(direction)` in the Sense phase.

## Your defence questions

---

**How do you prevent collisions?** Each tick, every vehicle receives a sensor snapshot. It limits speed by the closest same-lane free space or intersection stop. Vehicles also wait for occupied intersections and blocked downstream lanes.

**What is a tick?** One complete simulation update. At 30 Hz that's ~33 ms of simulated update scheduling.

**Why use synchronisation?** The sim and Swing drawing run on different threads. Locking the network makes a frame show a complete state rather than a partial update.

**Why is the simulation random but repeatable?** Normal launches use `SecureRandom` to make a seed. Passing `--seed` supplies the same seed, so spawning and driver slowdown repeat deterministically.

**Why the Observer pattern?** Engine informs the UI after each tick without depending on drawing details. Swap the display for a headless logger and nothing else changes.

## Patterns you're defending

---

- **Observer** - `SimulationObserver` / `SimulationDisplay`
- **Record** - `SensorReading`
- **Streams / lambdas** - `Statistics` (HUD chart aggregation)
- **Sealed hierarchies** - you consume `Vehicle`, `Intersection`, `SimulationCommand` polymorphically
- **Command** - `SimulationCommand` (sealed) + `Pause` / `Resume` / `Step` / `Reset` / `SpawnOne` / `SetTickRate`

## What to skip in the main PDF

---

- Section 5 (Lane geometry) - Pair B's domain
- Section 7 (Vehicle subclasses in detail) - Pair C's
- Section 9 (Adaptive green extension math) - Pair D's
- Section 4 (grid.txt format) - Pair B's; you just call the loader